

Adversarial Co-Evolution of RL and LLM Agents in Gin Rummy: A Gold-Standard Study of What Makes a Lightweight Agent Strong

Nima Kelidari Mahdi Salmani Mohammadsaeed Haghi
{kelidari, salmanis, haghim}@usc.edu
University of Southern California

Abstract

Training a reinforcement-learning (RL) agent for a card game with hidden information runs into the *opponent bottleneck*: an agent is only as good as the opponents it practises against. We study this in Gin Rummy, used here as a clean example and not as the goal. We build a full framework with five parts: an action-masked PPO/TRPO learner, a hand-coded *gold-standard* (perfect) opponent used only as a measuring stick, a distributed server that lets a 7B language model (LLM) play inside the game, an opponent-curriculum system, and a browser game. We then test almost every reasonable way to make the agent stronger, judged on one number: win-rate against the perfect player. Across 100+ controlled runs we find three things. First, the gold standard beats every learned agent (70 to 99%) yet gins in under 2% of games, winning by knocking with low deadwood, and an RL agent learns the same habit even when a gin pays $3\times$ a knock. Second, imitation learning (DAGger), dense short-term reward shaping, and training against a live LLM all fail to beat plain curriculum RL, each for a clear reason. Third, the ideas that *do* help, namely a trust-region algorithm (TRPO), a knock-first reward, a rising opponent curriculum, warm-starting, and keeping the best checkpoint, stack together and push a small agent from the $\sim 30\%$ self-play champion to **34%** against perfect play. We report the engineering and an honest look at why the popular hybrid methods do not beat pure RL here. The framework and these lessons are not specific to Gin Rummy. They carry over to other two-player games and to RL plus LLM agent systems in general.

1 Introduction

Reinforcement learning is good at making a sequence of decisions, but it needs a lot of data and it depends heavily on its opponents. Train against a weak player and the agent learns to beat weakness rather than to play well. Strong opponents are exactly what is hard to get: hand-built solvers are specific to one game, and humans do not scale. Large language models (LLMs) look like a shortcut because they bring broad, common-sense strategy with no training. This project asks, in the simple testbed of Gin Rummy, **how close a small, fast RL agent can get to perfect play, and which training ideas actually move it there**. We pick Gin Rummy because it is easy to score yet hard to play well, but the questions and the answers are general.

Contributions. (1) A reusable framework that ties together masked-PPO/TRPO RL, a *perfect* benchmark agent, a distributed LLM-serving stack, an opponent curriculum, and a web game (Sec. 2). (2) A gold-standard benchmark and one clean result: the perfect player almost never gins, and an RL agent learns the same thing no matter the reward (Sec. 3, 4.6). (3) A side-by-side study of about ten training methods on one number, win-rate against the perfect player, with the *reason* each one works or fails (Sec. 4, Fig. 3), including honest negatives for DAGger, dense rewards, learned embeddings, and a live LLM in the loop. (4) A recipe that stacks the helpful ideas to reach 34% against perfect play (Sec. 4.7, 5).

2 The Framework

Environment and learner. We use PettingZoo/RLCard `gin_rummy_v4`. The observation is four 52-card planes (hand, top discard, known picks, the rest). Our `MaskedGinRummyPolicy` sets illegal-action scores to $-\infty$, so the agent only ever picks a legal move, and it acts by choosing the highest-probability *legal* move (no random fallback). **Gold standard.** A hand-coded perfect player works out the exact best melds with RLCard’s solver and knocks the moment it should. It is the measuring stick and *never* trains the RL agent. **Distributed LLM.** One rollout asks the opponent tens of thousands of questions, and at 0.5 to 3s per 7B call a simple loop takes hours, so we keep serving separate from training. A master node balances load over self-registering GPU workers behind a suit-symmetry cache, using the normal Ollama API. One infrastructure result stood out: putting 7B weights on scratch (BeeGFS) instead of home storage cuts worker load from about 28min to about 27s, a $62\times$ speedup, and a 14-worker pool handled about 32 questions per second. **Curriculum and game.** A scheduler ramps opponents (random, then past selves, then strong models), and a zero-dependency browser client lets people play any agent. Fig. 1 shows how the parts connect.

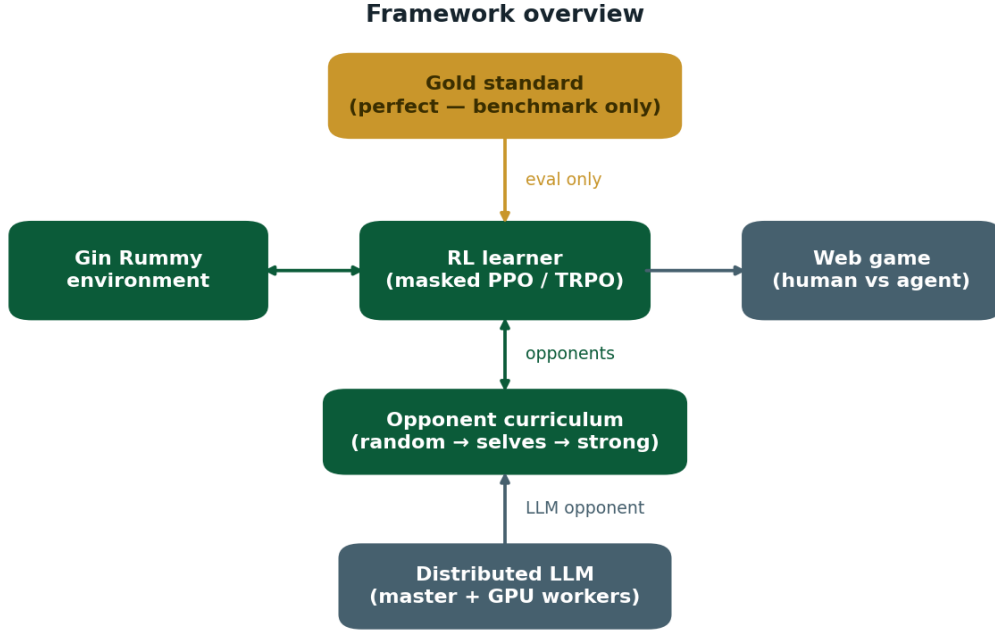


Figure 1: System overview. The learner trains in the environment against a curriculum of opponents (and, in dedicated runs, the LLM); the gold standard is used for *scoring only*.

3 The Gold-Standard Benchmark and a Surprise

To grade everything on one fair scale we built a perfect player and ran it against the whole set of trained agents (Fig. 2, 600 games each). It wins 70.2% against the self-play champion and 70 to 99% against the rest. The surprise is how it wins: the perfect player **gins only 0.7 to 1.7% of the time**, even though gin scores the most points. It wins by *knocking early with low deadwood*. That goes against the natural guess that chasing gin is good, and it changes how we read every reward experiment below.

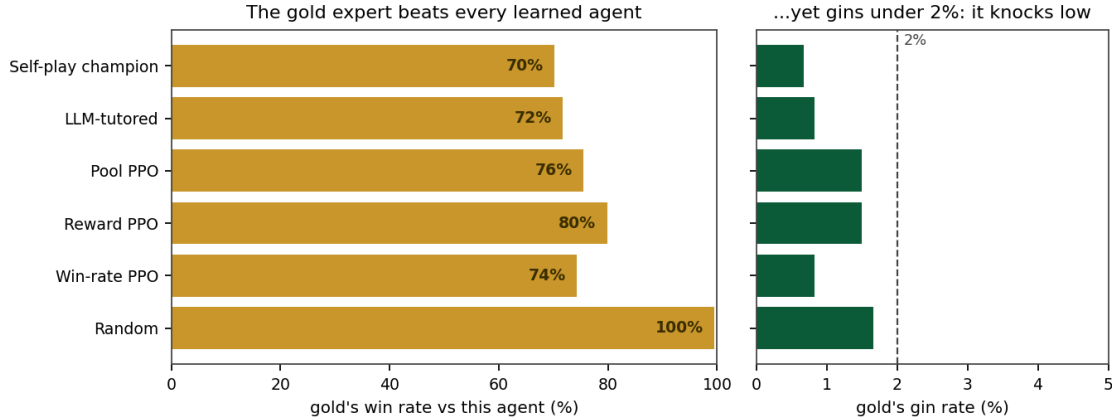


Figure 2: The perfect player beats every learned agent (left) yet gins under 2% of the time (right): it wins by low-deadwood knocking, not by chasing gin.

4 What We Tried: A Side-by-Side Study

We tested almost every reasonable training method on the same number, best win-rate against the perfect player (Fig. 3), and for each we give the reason it lands where it does.

4.1 RL backbone and reward shaping

Masked PPO maxes out against random opponents: an eight-setting sweep lands every run in the 98.3 to 99.6% band (you cannot tell them apart at 1000-game eval), and a larger set of 40+ reward-shaping runs shows the gin-to-knock reward ratio is a real dial. A knock-first reward gives 97% knocks (win 99.4%), while a gin-first reward gives 22% gins

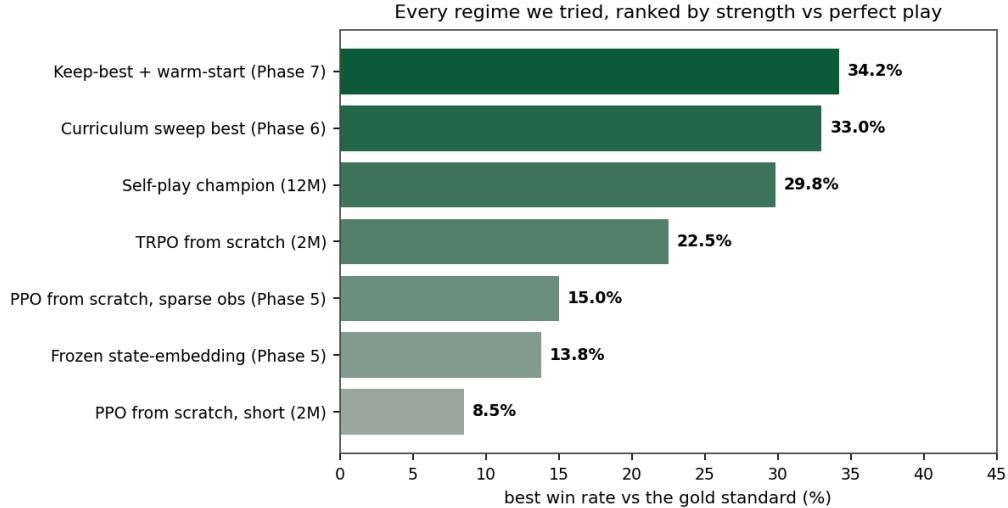


Figure 3: Every method we tested, ranked by measured win-rate against the perfect player.

(win 98.3%). *Reason:* random play is too weak to teach strategy, so the easy number maxes out, and the reward ratio sets how much risk the agent takes. This is why we need stronger opponents and a sensible reward.

4.2 Self-play and curriculum

Self-play is a free, rising curriculum: a 3M-step self-play agent beats its own parent 61% of the time. But a pool of past selves with no guidance *broke down* after about 10M steps. *Reason:* self-play makes difficulty for free, but it can chase circular strategies (where A beats B, B beats C, and C beats A), so the schedule and the mix of opponents matter. That motivates the controlled sweep in Sec. 4.6.

4.3 Why the hybrid methods fail (honest negatives)

Live LLM in the loop. Mid-size LLMs (Qwen2.5-7B, gpt-oss-20b) play real Gin Rummy with a chain-of-thought prompt (100% legal moves) and even beat our self-play agent 3 to 2 in short matches. But at 9 to 27s per move they are far too slow for the millions of moves RL needs, and even with the distributed stack, training this way would take weeks. Vision-LLMs failed outright (about 10% legal moves): this is a text task. **Imitation (Dagger).** Copying an expert’s moves one at a time collapses to almost no wins. *Reason:* the student copies moves in familiar spots but never learns the thinking behind them (like tracking the opponent), so it falls apart in new situations. Copying a move is not the same as understanding why. **Dense rewards.** Rewarding every step (horizon $L=2, 5$) makes the agent short-sighted: $L=2$ shows no learning and $L=5$ stops improving after about 500k steps, farming instant points instead of trying to win. *Reason:* dense shaping pulls the agent away from the real, sparse goal. The rule that follows: we let the LLM or expert *inform our design*, but never supervise the agent directly.

4.4 Algorithm: PPO vs. TRPO

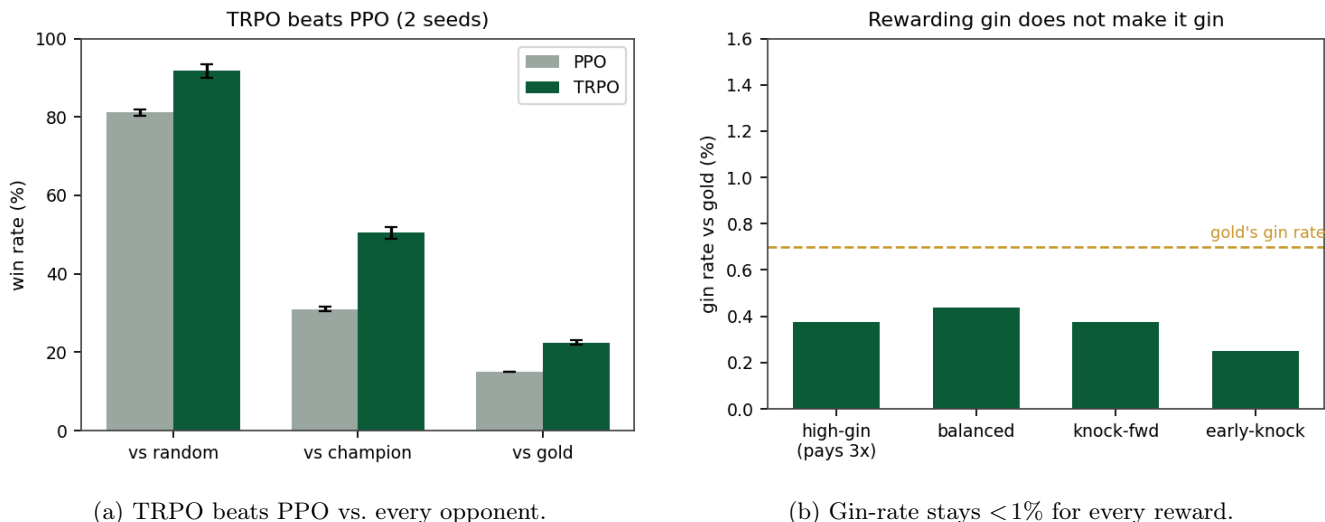
With the same masking, curriculum, and reward, TRPO beats PPO at 2M steps on every opponent and both seeds: 50.5% vs. 31.0% against the champion and 22.5% vs. 15.0% against gold (Fig. 4a). *Reason:* TRPO takes smaller, safer steps when it updates the policy, which fits a setting where rewards are rare and the opponent keeps changing. (GRPO and DPO do not apply here: they align language models and have no per-move game version.)

4.5 State representation

We squeezed the big, sparse observation into a small dense 20-d vector two ways: one that learned which game states are close together, and one where an LLM judged how similar two states are. Used frozen as the agent’s input, both *lose* to the raw sparse input against gold (6 to 14% vs. 15%). *Reason:* a fixed, squeezed vector that is good at “are these two states similar” still throws away detail the agent needs, and neither a bigger size nor un-freezing fixed it.

4.6 The curriculum sweep and the key finding

A 30-run sweep that changes one thing at a time across algorithm, reward, and curriculum lands every recipe near champion strength (~ 30 to 33% vs. gold), but it gives one clean, solid result (Fig. 4b): **the gin-rate stays under 1% no matter the reward**. Even paying 3 $\times$ for a gin does not raise it above the balanced baseline. *Reason:* like the perfect player, the agent works out that chasing gin loses, and you cannot pay it into the bad habit. The reward that does help is the opposite, a small “knock earlier” penalty that gives the shortest games and the best play. Fig. 5 shows the mechanism behind the numbers: win-rate against the champion climbs as the curriculum swaps in tougher opponents (random, then pool, then self, then strong), and the late dip on one run is the drift that keep-the-best fixes



(a) TRPO beats PPO vs. every opponent.

(b) Gin-rate stays <1% for every reward.

Figure 4: Algorithm and reward studies (2 seeds each).

(Sec. 4.7).

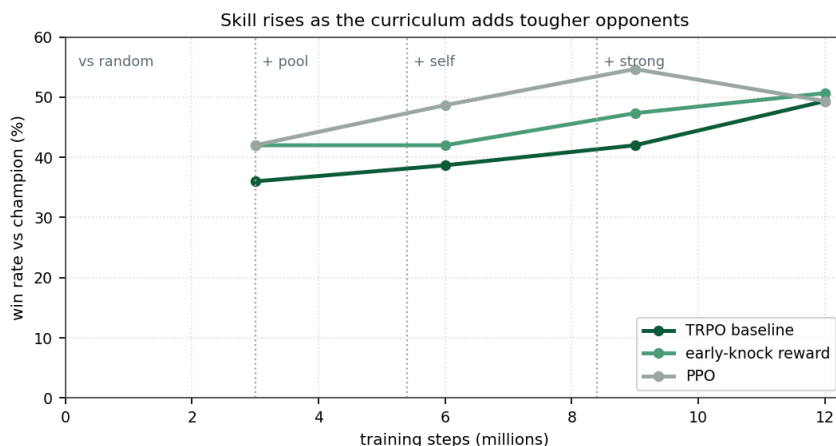


Figure 5: Skill (win-rate vs. the champion) rises through the curriculum stages; the late dip motivates best-checkpoint selection.

4.7 Keep-the-best and warm-start (the recipe)

A final series stacks the ideas that help: *warm-start* from the champion, the knock-first reward plus a small *deadwood-reduction* reward (a nudge toward the optimal style that does not change which strategy is best, computed from the agent’s own public hand), a curriculum seeded with the strongest earlier agents, and, most important, *keeping the best checkpoint* (training drifts past its peak, so we evaluate every 1M steps and ship the best one). Checked over 2000 games, the strongest agent reaches $34.2\% \pm 2.1\%$ vs. gold and 51.4% vs. the old champion, our best, up from the champion’s $\sim 30\%$.

5 What Mattered, and the Bottom Line

Stacking the helpful ideas took a small agent from the champion’s $\sim 30\%$ to 34% against *perfect* play (Fig. 6) and gave one reusable result (the best play rarely gins). The honest ceiling is the mid-30s percent: tuning reward, algorithm, and opponents tops out there. None of this is specific to Gin Rummy. The same recipe, a strong reference to grade against, a rising ladder of opponents, a reward that copies strong play, and best-checkpoint selection, applies to other two-player games and to RL plus LLM agent systems in general.

6 Conclusion and Future Work

We built the framework, the benchmark, and the side-by-side study to answer, fairly, how strong a small RL agent can be in Gin Rummy and what makes it so. Plain curriculum RL, sharpened with a trust-region algorithm, a knock-first reward, warm-starting, and best-checkpoint selection, is the winner, and the popular hybrid shortcuts (imitation, dense

Idea	Verdict	Why
Keep the best checkpoint	helps	training drifts past its peak; saving the best recovers 2 to 3 pts
Warm-start from the champion	helps	start strong, then improve
TRPO over PPO	helps	smaller, safer steps suit rare rewards and a changing opponent
Reward knocking, not gin	helps	copies the perfect player's low-risk style
Rising opponent curriculum	helps	always a fair but harder challenge
Pay more for gin	no effect	the agent refuses the bad habit at any reward
Learned state embeddings	fails	a fixed, squeezed input throws away useful detail
Imitation (Dagger)	fails	copies moves, not the thinking behind them
Dense short-term rewards	fails	short-sighted: greedy for points, blind to winning
Live LLM in the loop	fails	strong, but far too slow for millions of moves

Table 1: Which ideas moved the needle against the perfect player, across 100+ runs.

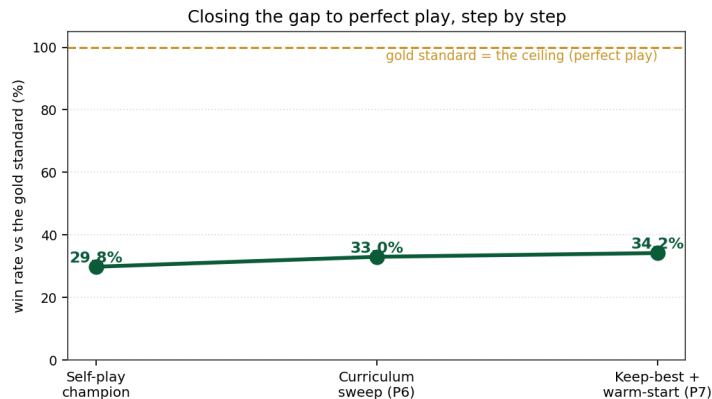


Figure 6: Closing the gap to perfect play, step by step.

reward, a live LLM in the loop) each fail for a clear, named reason. Going clearly past the mid-30s ceiling against perfect play almost certainly needs a *different kind* of method, such as search and planning (the kind that beat the best humans at poker, counterfactual-regret minimization) or an agent that remembers the whole game so far, rather than more reward tuning. The framework, the benchmark, and the distributed LLM data pipeline are the reusable base for that next step (for example, offline RL on a large LLM-vs-LLM dataset), and they apply well beyond this one game. All figures rebuild from saved JSON, and live training curves are on Weights & Biases.

References

- [1] J. Schulman et al. Proximal Policy Optimization Algorithms. *arXiv:1707.06347*, 2017.
- [2] J. Schulman et al. Trust Region Policy Optimization. *ICML*, 2015.
- [3] S. Huang and S. Ontañón. A Closer Look at Invalid Action Masking in Policy Gradient Algorithms. *arXiv:2006.14171*, 2022.
- [4] S. Ross, G. Gordon, D. Bagnell. A Reduction of Imitation Learning to No-Regret Online Learning (Dagger). *AISTATS*, 2011.
- [5] N. Brown and T. Sandholm. Superhuman AI for multiplayer poker. *Science* 365(6456):885–890, 2019.
- [6] J. Terry et al. PettingZoo: Gym for Multi-Agent Reinforcement Learning. *NeurIPS*, 2021.
- [7] D. Zha et al. RLCard: A Toolkit for Reinforcement Learning in Card Games. *arXiv:1910.04376*, 2019.
- [8] A. Raffin et al. Stable-Baselines3. *JMLR* 22(268):1–8, 2021.
- [9] Qwen Team. Qwen2.5 Technical Report. *arXiv:2412.15115*, 2024.